

1. Concept of Programming

1.1 Programming Language

Programming language: A programming language is a language that consists of a set of instructions provided by the user to tell a computer what task to perform and how to perform it.

Examples: Python, Java, PHP, C, C++, JavaScript, etc.

1.2 Programming

Programming: Programming is the process of providing detailed instructions to a computer, step by step, to perform a specific task or solve a problem.

1.3 Programmer

Programmer: A programmer is a person who writes, tests, and maintains computer programs.

1.4 Syntax

Syntax: Syntax is the set of rules that determines how commands and statements must be written in a programming language.

A syntax error occurs when a statement does not follow the rules of the programming language.

1.5 Types of Programming Languages

Type	Meaning	Examples
High-level language	A human-friendly language that uses understandable words and simpler syntax. It is easier to write, read, debug, and maintain.	Python, Java, C++, PHP
Low-level language	A language closer to the hardware and machine instructions. It provides more direct control over computer resources but is harder for humans to read.	Machine language, Assembly language

2. Programming Tools

2.1 Algorithm

An algorithm is a step-by-step set of instructions designed to solve a specific problem or perform a particular task. An algorithm is like a recipe for computer programming: each step must be clear and arranged in the correct order. It generally begins with START and ends with STOP/END.

Characteristics of an Algorithm

- Written in simple, general language.
- Steps are arranged in a logical order.
- Begins with START and ends with STOP/END.
- Solves a particular problem or performs a particular task.

2.2 Flowchart

A flowchart is a pictorial or graphical representation of an algorithm. It uses standard symbols to show different steps and the flow of control.

Each symbol has a standard meaning, so a flowchart can visually represent the structure and sequence of an algorithm.

Symbol / Name	Shape	Meaning / Use
Terminator	Oval	Shows the START or END of a process.
Input / Output	Parallelogram	Represents input received by the program or output produced by it.
Process	Rectangle	Represents a calculation, instruction, or processing step.
Flow line	Arrow	Shows the direction or sequence of flow.
Decision	Diamond	Represents a condition or decision such as Yes/No or True/False.
Connector	Circle	Connects different parts of a flowchart, especially when the flow continues elsewhere.

3. Coding, Testing and Debugging

3.1 Coding

Coding is the process of writing instructions in a programming language to create software, applications, or solutions to problems.

3.2 Testing

Testing is the process of running a program and checking whether it works as expected and produces the correct result.

3.3 Debugging

Debugging is the process of finding the source of a program error, understanding why it occurs, and correcting it.

Coding, testing, and debugging work together to produce software that is reliable, correct, and functions smoothly.

Stage	Main purpose
Coding	Write the program instructions.
Testing	Find errors and verify expected behaviour.
Debugging	Locate and correct errors.

4. Compiler and Interpreter

4.1 Compiler

A compiler translates the entire source program into machine code or another lower-level form before the program is executed.

Example languages commonly associated with compilation: C, C++, and Java (Java source is compiled to bytecode).

4.2 Interpreter

An interpreter translates and executes program instructions during execution, generally one statement or unit at a time.

Example languages commonly associated with interpretation: Python and JavaScript. Some modern implementations use a mixture of interpretation and compilation techniques.

Compiler	Interpreter
Translates the program before execution.	Translates/executes during program execution.
Usually produces an executable or intermediate form.	Usually executes from the source or intermediate representation.

Reports many errors after compilation.	Often stops at or reports an error when the problematic statement is reached.
Compiled program generally runs faster after successful compilation.	Execution may involve additional translation work.
Examples: C, C++, Java (compile to bytecode).	Examples: Python, JavaScript (implementation-dependent).

5. Introduction to Python

Python is a high-level programming language introduced by Guido van Rossum in 1991. It became popular because its syntax is relatively simple, readable, and versatile.

Python can be used to solve many types of problems and to create applications, automation scripts, games, and data-driven programs.

5.1 Applications of Python

- Data science
- Web development
- Machine learning
- Artificial intelligence
- Data analysis
- Automation and scripting
- Game development
- Web scraping
- Cybersecurity

5.2 Features of Python

Feature	Short explanation
Easy to read and write	Simple, readable syntax.
Versatile	Used in many different fields.
Beginner-friendly	Suitable for learning programming fundamentals.
Extensive standard library	Provides many built-in modules and tools.
Rich ecosystem	Large collection of frameworks, libraries, and community resources.

6. Comments and Keywords in Python

6.1 Comments

A comment is a note written in source code for explanation or documentation. Python ignores comments during normal program execution.

a. Single-line Comment

A single-line comment begins with # and continues to the end of that line.

```
# This is a single-line comment
print("Hello Python")
```

b. Multi-line Comment

Python has no separate multi-line comment keyword. A common teaching/documentation method is to use triple-quoted strings (""" ... """ or ''' ... ''') when the text is not used as a value. For exam purposes, these are often described as multi-line comments/docstrings.

```
"""
This is a multi-line
comment-style block.
"""
print("Python")
```

6.2 Python Keywords

A keyword is a reserved word in Python that has a special meaning and cannot normally be used as an ordinary identifier.

Common Python keywords include:

False, None, True, and, as, assert, async, await, break, case, class, continue, def, del, elif, else, except, finally, for, from, global, if, import, in, is, lambda, match, nonlocal, not, or, pass, raise, return, try, while, with, yield.

7. Input/Output Statements and String Formatting

I/O statement: Input/Output statements allow a program to receive data from the user and display information on the screen.

7.1 print() Function

The print() function displays information on the screen.

```
print("Hello Python")
```

7.2 input() Function

The input() function accepts data from the user through the keyboard. By default, it returns the entered value as a string.

```
number = input("Enter number: ")
print(number)
```

7.3 String Formatting

String formatting means inserting values into a message or arranging text so that it contains the required information.

1. Old-style Formatting (%)

```
name = "Ram"
age = 25
print("My name is %s and I am %d years old." % (name, age))
# Output: My name is Ram and I am 25 years old.
```

2. New-style Formatting: str.format()

```
name = "Sita"
age = 30
print("My name is {} and I am {} years old.".format(name, age))
# Output: My name is Sita and I am 30 years old.
```

3. Formatted String Literals (f-strings)

```
name = "Shiva"
age = 35
print(f"My name is {name} and I am {age} years old.")
# Output: My name is Shiva and I am 35 years old.
```

8. Data Types, Identifiers and Variables

8.1 Data Type

A data type classifies data according to its nature and tells Python what kind of value is being stored.

Data type	Description	Example
int	Whole numbers, including negative and positive integers.	-10, 0, 25
float	Numbers containing a decimal point.	3.14, 2.5, -0.75
str	Text made of letters, numbers, spaces, or special characters enclosed in quotation marks.	"Hello", "Grade 9"
bool	Logical value containing either True or False.	True, False

8.2 Identifier

An identifier is the name given to a program unit such as a variable, function, class, or other entity.

Rules for Identifiers

- An identifier must start with a letter (A-Z or a-z) or an underscore (_).
- It cannot start with a number.
- It can contain letters, digits, and underscores only.
- Python is case-sensitive, so name and Name are different identifiers.
- An identifier cannot be a Python keyword.

Valid identifiers	Invalid identifiers	Reason
age	2age	Cannot start with a number.
student_name	student-name	Hyphen is not allowed.
_total	class	class is a keyword.
firstName	first name	Space is not allowed.

8.3 Variable

A variable is a named container/reference used to store a value in a program. In Python, a variable is created when a value is assigned to a name.

```
age = 18
height = 5.8
fname = "Ram"
is_student = True
```

Python does not require a separate command to declare a variable before assigning a value.

8.4 Typecasting

Typecasting is the process of converting a value from one data type to another.

a. Implicit Typecasting

Python automatically converts a value to a compatible type when necessary, without an explicit conversion command.

```
a = 5      # int
b = 2.5    # float
c = a + b  # c becomes 7.5, a float
```

b. Explicit Typecasting

The programmer explicitly converts a value using functions such as `int()`, `float()`, `str()`, or `bool()`.

```
x = "25"
age = int(x)
print(age + 5) # 30
```

9. Operators in Python

9.1 Arithmetic Operators

Operator	Description	Example
+	Addition	$5 + 2 = 7$
-	Subtraction	$5 - 2 = 3$
*	Multiplication	$5 * 2 = 10$
/	Division	$5 / 2 = 2.5$
%	Remainder / Modulus	$5 \% 2 = 1$
**	Exponent / Power	$5 ** 2 = 25$
//	Floor division	$5 // 2 = 2$

9.2 Relational / Comparison Operators

Operator	Description	Example
==	Equal to	$5 == 5 \rightarrow \text{True}$
!=	Not equal to	$5 != 3 \rightarrow \text{True}$
>	Greater than	$5 > 3 \rightarrow \text{True}$
<	Less than	$5 < 3 \rightarrow \text{False}$
>=	Greater than or equal to	$5 >= 5 \rightarrow \text{True}$
<=	Less than or equal to	$3 <= 5 \rightarrow \text{True}$

9.3 Logical Operators

Operator	Description	Example
and	True only when both conditions are True	$(5 > 2) \text{ and } (4 > 1) \rightarrow \text{True}$
or	True when at least one condition is True	$(5 < 2) \text{ or } (4 > 1) \rightarrow \text{True}$
not	Reverses a Boolean result	$\text{not True} \rightarrow \text{False}$

9.4 Assignment Operators

Operator	Meaning	Example
=	Assign value	$x = 5$
+=	Add and assign	$x += 2$
-=	Subtract and assign	$x -= 2$
*=	Multiply and assign	$x *= 2$
/=	Divide and assign	$x /= 2$
%=	Remainder and assign	$x \% = 2$
**=	Exponent and assign	$x ** = 2$
//=	Floor-divide and assign	$x //= 2$

10. Expressions and Operands

An expression is a combination of values, variables, operators, and functions that Python evaluates to produce a result.

An operand is a value or variable on which an operator performs an operation.

Expression type	Python example	Meaning
Mathematical expression	<code>a = 5 + 3 * 2</code>	Performs arithmetic operations.
Text expression	<code>full_name = first + " " + last</code>	Combines text values.
Combined expression	<code>total = price * qty + tax</code>	Uses multiple operations together.
Algebraic expression	<code>2x + 5</code>	In Python, write <code>2*x + 5</code> .

11. Conditional Statements

A conditional statement is a decision-making structure that chooses what a program should do based on whether a condition is True or False.

11.1 if Statement

The if statement executes a block only when its condition is True.

Syntax

```
if condition:
    statement(s)
```

Example

```
if age >= 18:
    print("Adult")
```

11.2 if-else Statement

The if-else statement selects one of two blocks: one when the condition is True and another when it is False.

Syntax

```
if condition:
    statement(s)
else:
    statement(s)
```

Example

```
if marks >= 40:
    print("Pass")
else:
    print("Fail")
```

11.3 if-elif-else Statement

The if-elif-else structure checks multiple conditions in sequence and executes the first matching block.

Syntax

```
if condition1:
    statement(s)
elif condition2:
    statement(s)
```

```
else:  
    statement(s)
```

Example

```
if marks >= 80:  
    print("A")  
elif marks >= 60:  
    print("B")  
else:  
    print("C")
```

11.4 Nested if Statement

A nested if statement is an if statement placed inside another if statement.

```
if age >= 18:  
    if citizen:  
        print("Eligible")  
    else:  
        print("Not eligible")
```

12. Iteration and Loops

Iteration means repeating a particular task or set of statements until a specified condition or sequence is completed.

12.1 for Loop

A for loop is used to repeat a block of code for each item in a sequence or iterable.

Syntax

```
for variable in sequence:  
    statement(s)
```

Example

```
for i in range(1, 6):  
    print(i)
```

12.2 while Loop

A while loop repeatedly executes a block of code as long as its condition remains True.

Syntax

```
while condition:  
    statement(s)
```

Example

```
i = 1  
while i <= 5:  
    print(i)  
    i += 1
```

12.3 break, continue and pass

Statement	Purpose	Example
break	Immediately terminates the nearest loop.	if i == 3: break
continue	Skips the current iteration and continues with the next iteration.	if i == 3: continue
pass	Does nothing; used as a placeholder where a statement is syntactically required.	if x > 0: pass

12.4 for Loop vs while Loop

for loop	while loop
Best when the number of iterations or the sequence is known.	Best when repetition depends on a condition.
Automatically moves through a sequence/iterable.	Continues while the condition remains True.
Example: for item in list.	Example: while age < 18.

13. Python List

A list is an ordered, mutable collection of items written inside square brackets [].

```
fruits = ["apple", "banana", "mango"]
```

13.1 Features of a List

- List elements are indexed, starting from 0.
- A list can contain duplicate elements.
- Lists preserve the order in which items are stored.
- Lists are mutable, so elements can be changed.
- The del keyword can be used to delete a list element.

Example

```
numbers = [10, 20, 20, 30]
print(numbers[0])    # 10
numbers[1] = 25
del numbers[2]
print(numbers)       # [10, 25, 30]
```

13.2 List Length: len()

The len() function returns the number of elements in a list.

```
names = ["Ram", "Sita", "Hari"]
print(len(names))    # 3
```

13.3 Looping Through a List

```
fruits = ["apple", "banana", "mango"]
for fruit in fruits:
    print(fruit)
```

14. Python Dictionary

A dictionary is a mutable collection of data stored as key-value pairs. It is written inside curly braces {}.

```
student = {"name": "Ram", "age": 15, "grade": 9}
```

Here, name, age, and grade are keys, while Ram, 15, and 9 are their corresponding values.

14.1 Features of Dictionary

- Data is stored in key-value pairs.
- Duplicate keys are not allowed; a repeated key overwrites the earlier value.
- Dictionaries are mutable, so values can be changed, added, or removed.
- Dictionary values may be accessed using their keys.

14.2 Assigning / Changing a Value

```
student = {"name": "Ram", "age": 15}
student["age"] = 16
student["school"] = "ABC School"
```

14.3 Dictionary Length

```
student = {"name": "Ram", "age": 15, "grade": 9}
print(len(student)) # 3
```

14.4 Looping Through a Dictionary

```
student = {"name": "Ram", "age": 15}
for key in student:
    print(key, student[key])
```

14.5 List vs Dictionary

List	Dictionary
Stores items in a sequence.	Stores data as key-value pairs.
Uses index positions to access elements.	Uses keys to access values.
Written using [].	Written using { } for a dictionary.
Example: [10, 20, 30]	Example: {"a": 10, "b": 20}

15. Library and String Functions

15.1 String Functions / Methods

Function / Method	Syntax	Example	Use
center()	text.center(width)	"Hi".center(6)	Centers text within the given width.
upper()	text.upper()	"hello".upper()	Converts text to uppercase.
lower()	text.lower()	"HELLO".lower()	Converts text to lowercase.
len()	len(text)	len("Python")	Returns the number of characters/items.

15.2 Numeric Functions

Function	Syntax	Example	Use
sum()	sum(iterable)	sum([2, 3, 4])	Returns the total of numeric items.
abs()	abs(x)	abs(-7)	Returns the absolute value.
round()	round(x[, ndigits])	round(3.1416, 2)	Rounds a number to the given number of decimal places.
max()	max(iterable)	max([5, 9, 2])	Returns the largest value.
min()	min(iterable)	min([5, 9, 2])	Returns the smallest value.

15.3 math Module Functions

The math functions below are used through Python's math module. Import it before use:

```
import math
```

Function	Syntax	Example	Use
sqrt()	math.sqrt(x)	math.sqrt(25)	Returns the square root.
ceil()	math.ceil(x)	math.ceil(3.2)	Rounds upward to the nearest integer.
floor()	math.floor(x)	math.floor(3.8)	Rounds downward to the nearest integer.
factorial()	math.factorial(n)	math.factorial(5)	Returns n!, the factorial of n.
pow()	math.pow(x, y)	math.pow(2, 3)	Returns x raised to the power y.
sin()	math.sin(x)	math.sin(math.pi/2)	Returns sine of x (x in radians).
cos()	math.cos(x)	math.cos(0)	Returns cosine of x (x in radians).
tan()	math.tan(x)	math.tan(0)	Returns tangent of x (x in radians).
log()	math.log(x[, base])	math.log(10)	Returns the natural logarithm by default; an optional base can be supplied.

Note: the standard Python math module uses math.pow(), not math.power().